

A QUESTION FOR YOUR NEXT HIRING LOOP

Anyone can claim “production agentic AI.” How do you verify it in one hour?

Every screening call sounds identical now: LangGraph, RAG, guardrails, evals. The demo works. **The architecture is the question** — and a résumé can't answer it.

You already know where agent projects die — résumés don't show it



What your experience already tells you

Agent prototypes are easy; agent systems are not. Projects rarely die in the demo — they die in security review, in the audit, in month three when the architecture rots. **And the strongest candidates and the weakest ones describe their work in exactly the same vocabulary.**



The bind: every signal in a hiring loop is a proxy

Take-homes are toy-scale. System-design rounds are whiteboards — nothing compiles. References are curated. Portfolio repos are usually a notebook and a README. You're asked to commit a senior salary and a team's roadmap on signals you cannot diff, while the cost of a wrong call is measured in quarters, not weeks.

THE ANSWER

Interview me against a codebase you can audit.

The Trust-Native ReAct Agent Framework — a full-stack agent platform built solo in eight weeks. Four slides follow; each makes one claim a hiring loop normally can't check, and each is reproducible from the repository before we ever speak.

108K+

lines, Python
& TypeScript

2,500

test functions,
186 files

12

CI architecture
gates

146

commits,
Apr 17 – Jun 10

github.com/rajnishkhatri/AgentsFramework — clone it, run the gates, then decide if the hour is worth it.

Architecture boundaries here are CI failures, not conventions



Every framework has a layering diagram.



Nothing stops tomorrow's PR from importing across it — diagrams rot silently.

ORCHESTRATION

LangGraph topology & state — thin wrappers only

COMPONENTS

router · evaluator · plan builder — LangGraph imports forbidden

SERVICES

prompts · guardrails · governance · memory · tools

TRUST KERNEL

pure types & crypto — stdlib + Pydantic only, no I/O

dependencies flow downward only



12 boundary-gate test modules run as their own CI job

Dependency rules, service isolation, layer placement, read-only surfaces. An illegal import is a red build — the architecture cannot silently rot.

Verify: `pytest tests/architecture/ -q`

Tests outweigh the application code 2:1 — and CI never calls a model



Every candidate says they test.



Agent tests that hit live LLMs are flaky and expensive — so in most repos they quietly don't run.

2,500

test functions

in 186 files, organized by architectural layer

53.6K

lines of test code

vs 26.7K lines of Python application code

0

live LLM calls in CI

model calls mocked by marker scope on Python 3.13

1

cold-install CI job

hash-pinned requirements.lock proven from scratch



Property-based tests (Hypothesis) on the trust kernel; rejection tests written before acceptance tests for every guard; live-LLM evals isolated outside CI by marker — deliberately, not accidentally.

“Why did the agent do that?” has a signed, replayable answer



Guardrails are table stakes; everyone has them.



At incident time, an unrecorded decision is an unexplainable one — that's where regulated deployments stall.



Input guardrail — LLM-as-judge rejects injection & override attempts



Tool validators — deterministic gates: command allowlist, path sandboxing



Output guardrail — PII / API-key / prompt-leak scanning (regex + LLM)



Injection classifier — fourth line, in the governance layer



Every decision is captured and replayable

Black-box recordings (full I/O per run) · **phase logs** (per reasoning step) · **cryptographically signed agent-facts registry** (capabilities & policies, GCS-backed) · **6-module read-only explainability dashboard** — traces, decisions, guardrails, agents, compliance, live logs.

One engineer, eight weeks: runtime to deployable infrastructure



Most portfolio repos are a notebook and a README.



Production needs auth, memory, telemetry, IaC, CI — exactly where solo projects stop.

FRONTEND

27.7K lines of TypeScript

- Next.js 15 · React 19 · CopilotKit UI
- WorkOS auth · Zod · Tailwind v4
- 75 frontend test files

RUNTIME & MIDDLEWARE

26.7K lines of Python

- LangGraph · LiteLLM model routing
- Adapters: WorkOS JWT · Mem0 · Langfuse
- Postgres checkpointing · OpenAPI

INFRA & DELIVERY

20 Terraform files, 10 Rego policies

- Cloud Run · Cloudflare edge · Neon
- Docker images: agent, backend, search
- 4 GitHub Actions workflows

A 146 commits, Apr 17 → Jun 10, 2026 — the full git history is the timesheet.

THE ANSWER, RESTATED

Spend the interview hour where it can't be faked.

Probe these in the loop — I'll walk through any of them live:

Run the gates

pytest tests/architecture/ — then try to add an illegal import and watch it fail.

Audit the kernel

trust/ has zero framework dependencies. Ask me why authorization lives there and what signing buys.

Challenge the evals

~20 live-LLM and infra tests fail in the local cache, visibly. Ask me why they're excluded from CI on purpose.

Trace a decision

Pick any run; I'll replay it end-to-end through the explainability dashboard.

Fine print, on purpose: single-maintainer, pre-1.0, license pending. Nothing here is projected or extrapolated — every number is a static audit of the repository, and the failures are left where you can find them.